ITM SKILLS UNIVERSITY

School of Future Tech

Case Study Report

on

Eventify: Event Ticket Booking and Management System

by

Jui Tawde

Roll Number: 150096725149

ITM Skills University

Academic Year: 2026–2027

Index

1. Introduction to the Case Study 3

2. Problem Statement / Case Background (Abstract) 3

3. Problem Statement / Case Study Design 5

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study 7

5. Problem Statement / Case Study Implementation Details and Snapshots 11

6. Problem Statement / Case Study Results and Conclusion 20

7. References 23

1. Introduction to the Case Study

Event management involves organizing events, managing venues, handling ticket bookings, maintaining customer information, and tracking booking details. When these activities are performed manually, they can become time-consuming and may lead to errors in ticket availability, pricing, booking records, and cancellation management.

This case study focuses on the design and implementation of **Eventify: Event Ticket Booking and Management System**, a desktop application developed using Java and Java Swing. The application provides an interactive graphical user interface through which users can explore available events, search and sort event listings, select ticket categories, book tickets, view booking information, generate invoice details, and cancel bookings.

The system also includes an administrative interface that allows the administrator to view active bookings, access invoice details, cancel bookings, and add new events. The application maintains event, venue, customer, ticket, and booking information using Java classes and collection framework data structures.

The project demonstrates the practical application of object-oriented programming, graphical user interface development, event handling, data validation, sorting, searching, and data structure selection. It also illustrates how a modular Java application can organize related entities and perform common event-management operations through a single interface.

The primary objective of Eventify is to provide a simple, organized, and user-friendly solution for event ticket booking and management.

2. Problem Statement / Case Background (Abstract)

2.1 Background

Managing event registrations and ticket bookings requires accurate information about event schedules, venues, ticket categories, prices, available seats, customers, and booking status. A manual process may require maintaining separate records and repeatedly checking seat availability and payment amounts.

Such a process can result in duplicate effort, incorrect booking totals, difficulty locating records, and delays when handling cancellations or issuing booking confirmations.

A computerized event ticket booking system can simplify these activities by maintaining related information in an organized structure and automatically performing calculations and validation.

Eventify addresses this problem by providing a graphical application for event discovery, ticket booking, booking lookup, cancellation, invoice display, and administrative management.

2.2 Abstract

Eventify is a Java-based Event Ticket Booking and Management System developed using Java Swing. It provides separate user-facing and administrative functions through a graphical interface.

The user interface displays event information, including event name, category, date and time, venue, seat availability, and regular ticket price. Users can search for events using event names, categories, or venue names and sort the available events by date, name, or regular ticket price.

The booking module allows users to select an event, choose between Regular and Premium tickets, specify the ticket quantity, and enter customer details. The system validates required fields, performs basic email and phone-number checks, verifies seat availability, calculates the booking amount, and creates a booking record.

Eventify also supports booking lookup, invoice display, cancellation, and an administrative dashboard. The administrator can view active booking records and add new events with their category, schedule, venue capacity, and ticket prices.

Java collection framework classes such as ArrayList, LinkedList, HashMap, and TreeMap are used to organize application data. The project demonstrates how object-oriented programming and graphical interface components can be combined to develop a functional event-management application.

The current implementation stores its data in memory using Java collections rather than a permanent database. Therefore, the application is intended as a functional academic project and prototype rather than a production ticketing platform.

2.3 Objectives

The main objectives of Eventify are:

1. To develop a graphical desktop application using Java Swing.
2. To display event details in a structured tabular format.
3. To implement event searching and sorting.
4. To provide Regular and Premium ticket selection.
5. To validate customer information before confirming a booking.
6. To calculate ticket totals and check available seats.
7. To maintain booking records and provide invoice details.
8. To implement booking cancellation and update seat availability.
9. To provide administrative functionality for managing bookings and adding events.
10. To demonstrate the use of object-oriented programming and Java collection framework data structures.

3. Problem Statement / Case Study Design

Eventify is designed as a modular Java desktop application. The graphical user interface handles user interactions, the model classes represent the entities involved in event booking, and the DataStore class maintains the application's collections.

3.1 System Architecture

The application can be divided into the following logical components:

1. Graphical User Interface

The GUI is implemented using Java Swing components such as JFrame, JPanel, JLabel, JButton, JTable, JTextField, JComboBox, JSpinner, and JOptionPane.

The main Eventify class extends JFrame and coordinates the different application pages. CardLayout is used to switch between the Events, Book Tickets, My Bookings, Admin Dashboard, and Add New Event pages.

2. Model Classes

The application uses separate classes to represent the main entities:

- **Event:** Stores event-related information, including its name, category, schedule, venue, ticket prices, and booking-related seat information.
- **Venue:** Represents the venue and its seating capacity.
- **Customer:** Stores customer details such as name, email address, and phone number.
- **Ticket:** Represents the ticket category and its price.
- **Booking:** Connects the customer, event, ticket, booking ID, and quantity. It also maintains the booking's cancellation status.

3. Data Management

The DataStore class maintains the application's collections and initializes sample venues and events. It provides shared access to the event and booking information used by different parts of the interface.

4. Booking and Administrative Operations

The booking module validates information and creates booking records. The My Bookings page provides booking lookup, invoice display, and cancellation functions. The administrative interface provides booking management and event creation.

3.2 Main Modules of the System

A. Events Module

The Events page displays the available events in a table. The information includes event ID, event name, category, date and time, venue, seat availability, and regular ticket price.

The module supports searching by event name, category, or venue. A sorting control allows the user to arrange events by date, name, or regular ticket price.

B. Ticket Booking Module

The booking page provides controls for selecting an event, ticket category, and quantity. It also includes fields for the customer's full name, email address, and phone number.

The application displays the available seats and estimated booking amount based on the selected event, ticket category, and quantity.

When the user submits the booking, the system checks the entered information and seat availability before creating the booking record.

C. My Bookings Module

This module allows a user to enter a booking ID and retrieve the corresponding active booking. The user can display invoice details or request cancellation of the booking.

The cancellation process includes a confirmation prompt. After cancellation, the application updates the booking status, releases the associated seats, and refreshes the relevant interface.

D. Admin Dashboard

The Admin Dashboard displays active booking records in a table. The displayed information includes booking ID, event name, ticket category, quantity, amount, and booking status.

The administrator can load booking records, view invoice details, and cancel an active booking. Access to the administrative interface is controlled through the application's login interaction.

E. Add New Event Module

The Add New Event page provides fields for entering an event name, category, date and time, venue, venue capacity, regular ticket price, and premium ticket price.

The application checks that required text fields are completed and that the capacity and prices contain acceptable numeric values. It then creates the corresponding venue and event objects and adds the event to the application's data collections.

3.3 System Workflow

The general workflow of Eventify is as follows:

1. The application starts and initializes sample event and venue data if the event collection is empty.
2. The Events page displays the available events.
3. The user searches for an event or sorts the event list if required.
4. The user opens the booking page and selects an event, ticket category, and quantity.
5. The user enters customer information.

6. The application validates the details and checks whether enough seats are available.
7. If validation succeeds, the system creates the customer, ticket, and booking objects.
8. The booking is stored in the appropriate collections, and the event's booked-seat count is updated.
9. The application displays a booking confirmation and makes the booking available for lookup and invoice display.
10. The user or administrator can cancel an active booking, after which the application updates the booking status and releases the seats.
11. The administrator can also add new events through the administrative interface.

3.4 Data Flow

The main data flow begins with the event and venue information maintained in DataStore. The Events page retrieves event information from the event collection and displays it in a table.

When a booking is submitted, the application uses the selected event and customer inputs to create a Customer object, a Ticket object, and a Booking object. The booking is stored using its booking ID and is also added to the booking history collection.

The booking information is then used by the My Bookings and Admin Dashboard modules to display booking records and invoice details.

When a booking is cancelled, the application marks the booking as cancelled, removes it from the active booking map, decreases the event's booked-seat count, and refreshes the relevant displays.

The data remains in application memory while the program is running. The current implementation does not provide permanent database storage.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

4.1 Object-Oriented Programming

Eventify uses object-oriented programming to represent the main entities involved in event management.

Each model class has a specific responsibility. The Event class represents an event, the Venue class represents a location and its capacity, the Customer class represents a customer, the Ticket class represents a ticket category and price, and the Booking class connects the information required for a booking.

This approach organizes the program into smaller components and makes the relationships between different entities easier to understand and maintain.

The Eventify class coordinates the graphical interface and application operations, while DataStore provides shared access to the application's collections.

4.2 Java Swing and Event Handling

Java Swing is used to create the graphical user interface.

The application uses JFrame as the main window and JPanel to organize the different sections of the interface. JTable displays event and booking information in rows and columns.

JTextField accepts text input, JComboBox provides selectable options, and JSpinner allows the user to choose the ticket quantity. JButton components trigger operations such as searching, booking, cancellation, and adding events.

Event listeners respond to user actions. For example, an action listener executes the search method when the Search Events button is clicked. Selection changes in the event or ticket controls update the displayed availability and estimated total.

CardLayout is used to display different pages within the same main application window.

4.3 Java Collection Framework

The project uses multiple collection types for different data-management requirements.

1. ArrayList

ArrayList is used to maintain the lists of events, venues, and customers.

It allows the application to add new objects and iterate through the available records. It is suitable for displaying event information and creating filtered lists for searching and sorting.

2. LinkedList

LinkedList is used to maintain booking history. It keeps references to booking records, including records that have been cancelled, so the application can retain their history during the current execution.

3. HashMap

HashMap stores active bookings using the booking ID as the key and the Booking object as the value.

This allows the application to locate an active booking by its ID without manually searching through the entire booking list.

4. TreeMap

TreeMap stores events using a key formed from the event date and event ID. Its keys are maintained in sorted order, providing an ordered representation of events by that composite string key.

The application also uses a separate sorting method for the Events table.

4.4 Searching Algorithm

The search function accepts a search term entered by the user. The term is trimmed and converted to lowercase.

The application then iterates through the events stored in DataStore. For each event, it checks whether the event name, category, or venue name contains the search term.

Matching events are added to a result list and displayed in the event table. The Show All Events button clears the search field and displays the complete event list again.

This approach provides a straightforward linear search through the event collection.

4.5 Sorting Algorithm

Eventify supports sorting by date, event name, and regular ticket price.

A copy of the event list is created before sorting so that the original collection is not directly reordered by the table's sorting operation.

The application uses Java's Comparator utility with the list sorting method:

- `Comparator.comparing()` is used for event names and dates.
- `Comparator.comparingDouble()` is used for regular ticket prices.

The sorted list is then displayed in the event table.

4.6 Booking Validation and Amount Calculation

Before creating a booking, the application checks that an event is selected and that the customer's name, email, and phone fields are not empty.

It performs a basic email check by verifying that the email contains the `@` symbol. The phone number must contain at least ten characters.

The application also checks that the requested ticket quantity does not exceed the event's available seats.

The booking amount is calculated using the selected ticket category and quantity:

Total Booking Amount = Price per Ticket × Quantity

For a Regular ticket, the regular price of the selected event is used. For a Premium ticket, the premium price is used.

After successful validation, the application creates the relevant objects, generates a booking ID, updates the collections, and increases the event's booked-seat count.

4.7 Booking ID Generation

The application generates booking IDs beginning from 1001. Before returning an ID, the `nextBookingId()` method checks whether the ID already exists in the active booking map. If it does, the method increments the ID until it finds an unused value.

This prevents a new booking from overwriting an existing active booking with the same ID during the current execution.

4.8 Cancellation Algorithm

When a user or administrator requests cancellation, the application attempts to retrieve the booking using the supplied booking ID.

If the booking is missing or is already cancelled, the application displays an error message. Otherwise, it asks for confirmation.

After confirmation, the system marks the booking as cancelled, decreases the event's booked-seat count by the booking quantity, and removes the booking from the active booking map. The booking record remains in the history collection.

The application then refreshes the booking table and event information so that the released seats are reflected in the interface.

4.9 Technology Stack Used for the Case

Programming Language: Java

Graphical User Interface: Java Swing

Layout Management: BorderLayout, FlowLayout, GridBagLayout, BoxLayout, and CardLayout

Data Structures: ArrayList, LinkedList, HashMap, and TreeMap

Sorting Utilities: Comparator and Java list sorting

Table Management: JTable and DefaultTableModel

User Interaction: ActionListener, ChangeListener, and JOptionPane

Development Environment: Java Development Kit (JDK) and a Java-compatible IDE or editor

Data Storage: In-memory Java collections through the DataStore class

5. Problem Statement / Case Study Implementation Details and Snapshots

5.1 Implementation Details

Eventify is implemented as a Java Swing desktop application. Its main class, Eventify, extends JFrame and coordinates the graphical interface, navigation, event search, booking operations, invoice display, cancellation, and administrative functionality.

The application initializes sample data through the DataStore class when the event collection is empty. The initial data includes sample venues and events so that the user can explore the application without manually entering all the records.

The main interface is organized into separate pages using CardLayout. This allows the application to switch between the Events page, Book Tickets page, My Bookings page, Admin Dashboard, and Add New Event page without opening a separate main window for every operation.

The booking process connects the different model classes. After validation, the application creates the customer, ticket, and booking objects. It stores the active booking in the HashMap and adds the record to the LinkedList booking history. It also updates the booked-seat count for the selected event.

The invoice function retrieves the active booking and displays the booking ID, event name, ticket category, quantity, and total amount. The cancellation function updates the booking status and releases the associated seats.

The administrative interface allows the administrator to review active bookings, access invoice details, cancel bookings, and create new events. Newly created event information is added to the application's collections and becomes available for display and booking.

The following screenshots demonstrate the main features and working of the application.

5.2 Implementation Snapshots

Snapshot 1: Events Page

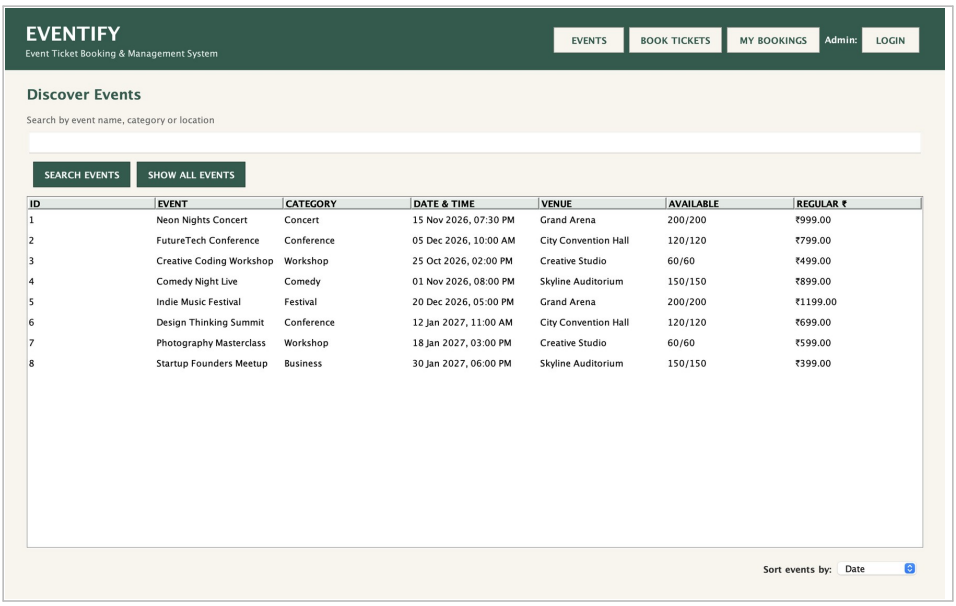


Figure 1: Eventify home page displaying available events, event details, venue information, seat availability, and ticket prices.

The Events page is the main screen displayed when the application starts. It presents event information in a tabular format, helping users compare the available events before making a booking.

Snapshot 2: Event Search and Sorting

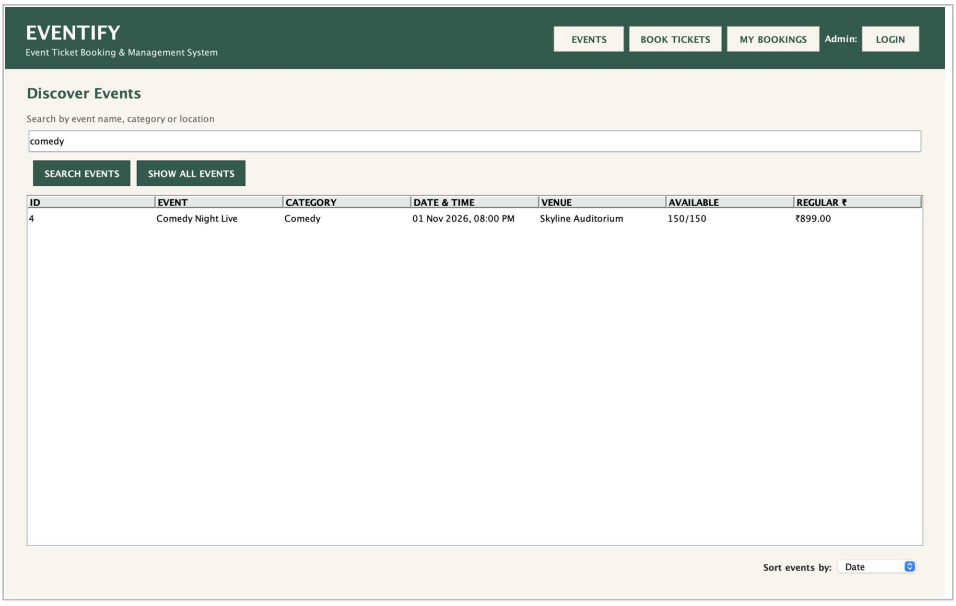


Figure 2: Event search and sorting functionality.

The search feature allows users to find events by name, category, or venue. The sorting control arranges the event list by date, name, or regular ticket price, making event discovery easier.

Snapshot 3: Ticket Booking Form

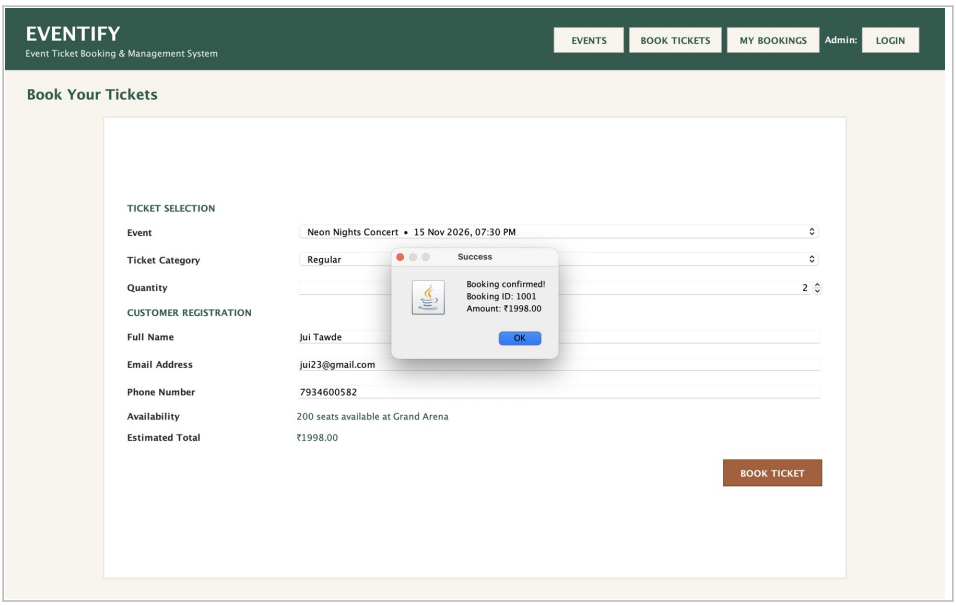


Figure 3: Ticket booking form with event selection, ticket category, quantity, customer details, seat availability, and estimated total.

The booking form collects the information required to make a reservation. Users select an event and ticket category, specify the quantity, and enter their customer details. The estimated total and available seats are displayed based on the current selection.

Snapshot 4: Successful Booking Confirmation

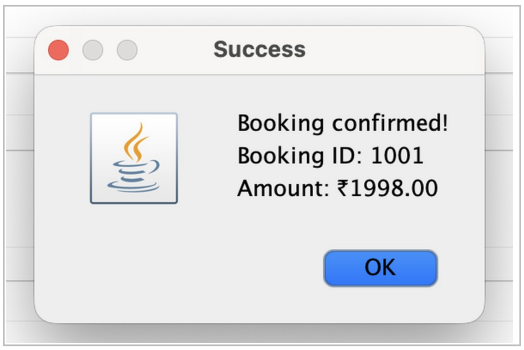


Figure 4: Successful ticket booking confirmation.

After the entered information passes validation and sufficient seats are available, the application creates the booking record and displays a confirmation message containing the booking ID and amount.

Snapshot 5: My Bookings Page

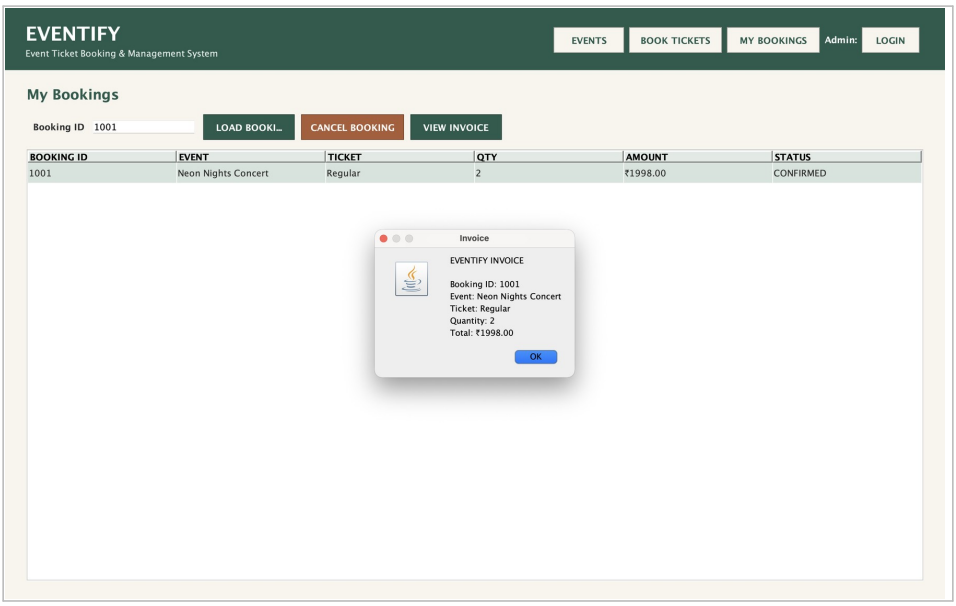


Figure 5: My Bookings interface for retrieving an active booking using its booking ID.

The My Bookings page allows the user to retrieve a booking by entering its ID. It provides access to the corresponding invoice details and cancellation functionality.

Snapshot 6: Invoice Details

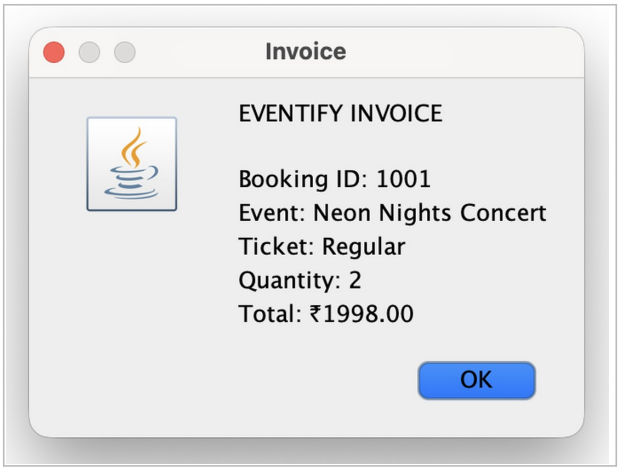
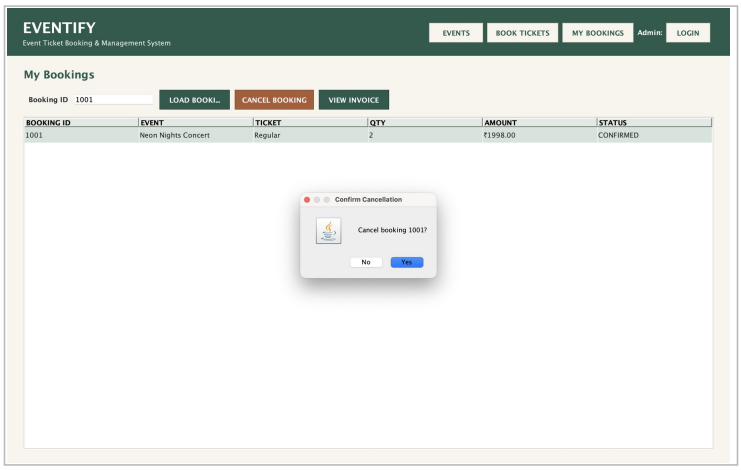


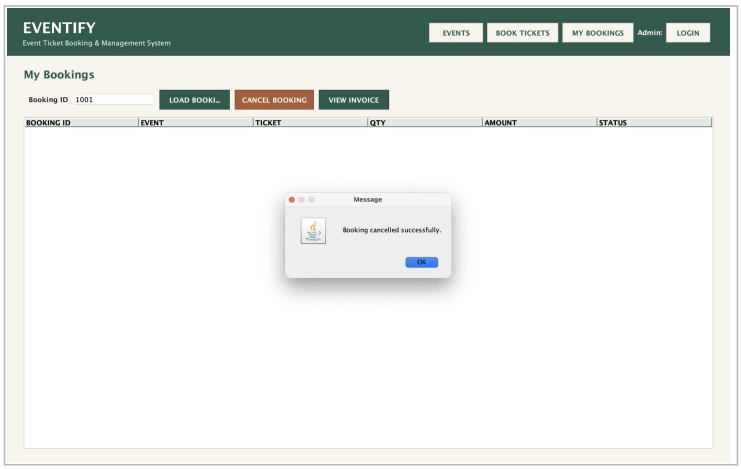
Figure 6: Invoice displaying booking details and the total ticket amount.

The invoice displays the booking ID, event name, ticket category, ticket quantity, and total amount. The amount corresponds to the selected ticket price multiplied by the number of tickets booked.

Snapshot 7: Booking Cancellation



(a) Cancellation confirmation prompt



(b) Cancellation success message

Figure 7: Booking cancellation process.

The cancellation feature asks the user to confirm the cancellation request. After confirmation, the booking is marked as cancelled, removed from the active booking map, and its seats are released. The updated information is reflected in the relevant application pages.

Snapshot 8: Administrative Login

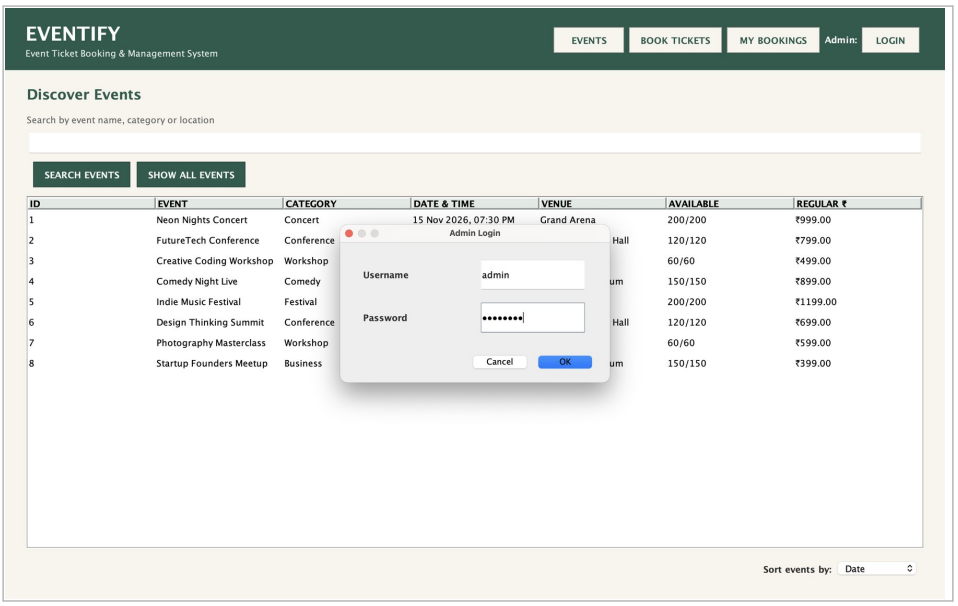


Figure 8: Administrative login interaction.

The application includes a login interaction for switching to the administrative interface. After successful administrative access, the interface provides navigation to the Admin Dashboard and Add New Event page.

Snapshot 9: Admin Dashboard

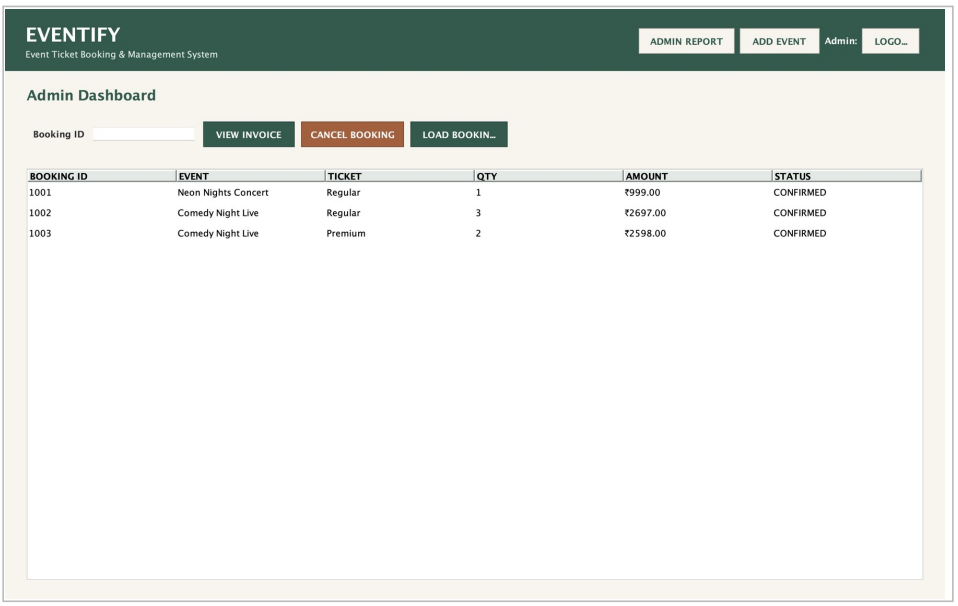


Figure 9: Admin Dashboard displaying active booking information.

The Admin Dashboard presents active bookings in a structured table. It includes the booking ID, event name, ticket category, quantity, amount, and status. The administrator can load the booking list, view invoice details, and cancel a booking using its ID.

Snapshot 10: Add New Event Form

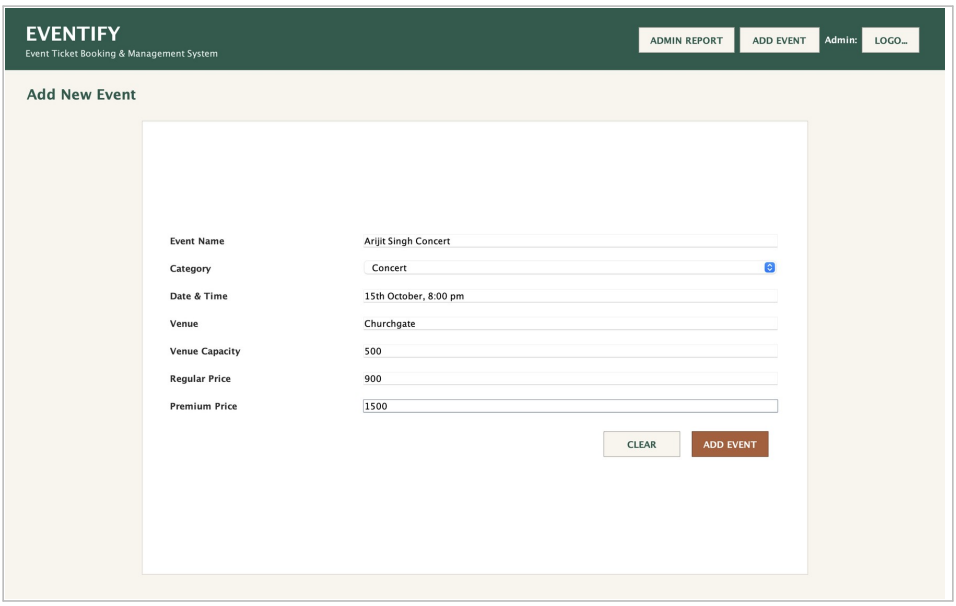
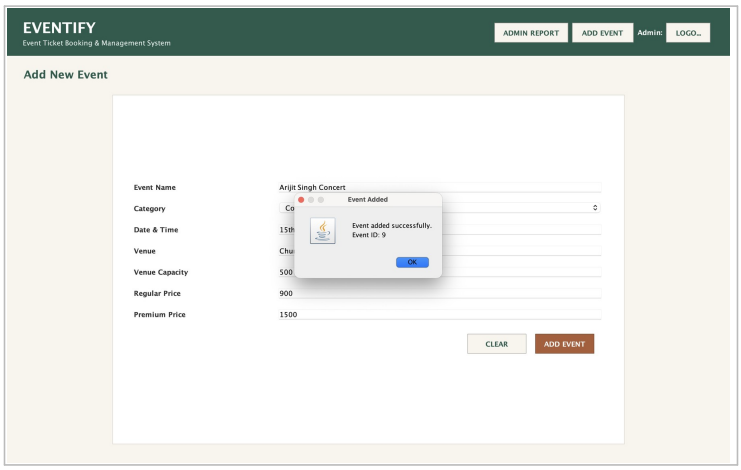


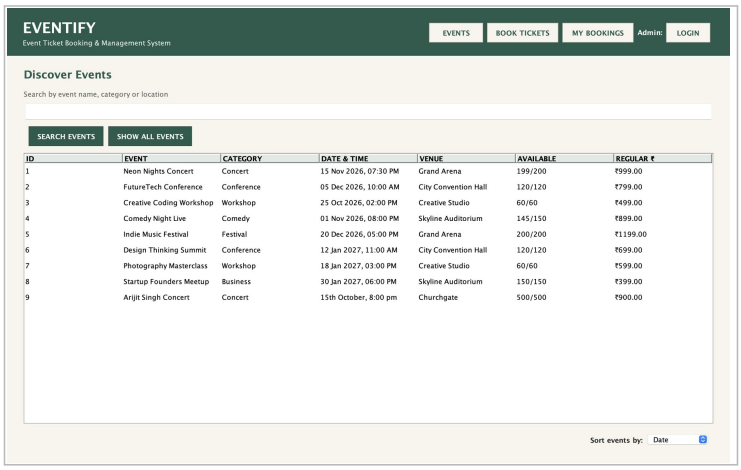
Figure 10: Add New Event form for entering event and venue information.

The Add New Event page allows the administrator to enter the event name, category, date and time, venue, venue capacity, and ticket prices. The form validates required fields and numeric values before creating a new event.

Snapshot 11: Newly Added Event



(a) Event added success message



(b) New event (ID 9) in the events table

Figure 11: Newly added event displayed in the event listing.

After an event is added successfully, it is inserted into the application's data collections and becomes available in the event listing. This demonstrates that the event-management functionality updates the application's data while it is running.

Snapshot 12: DataStore Implementation

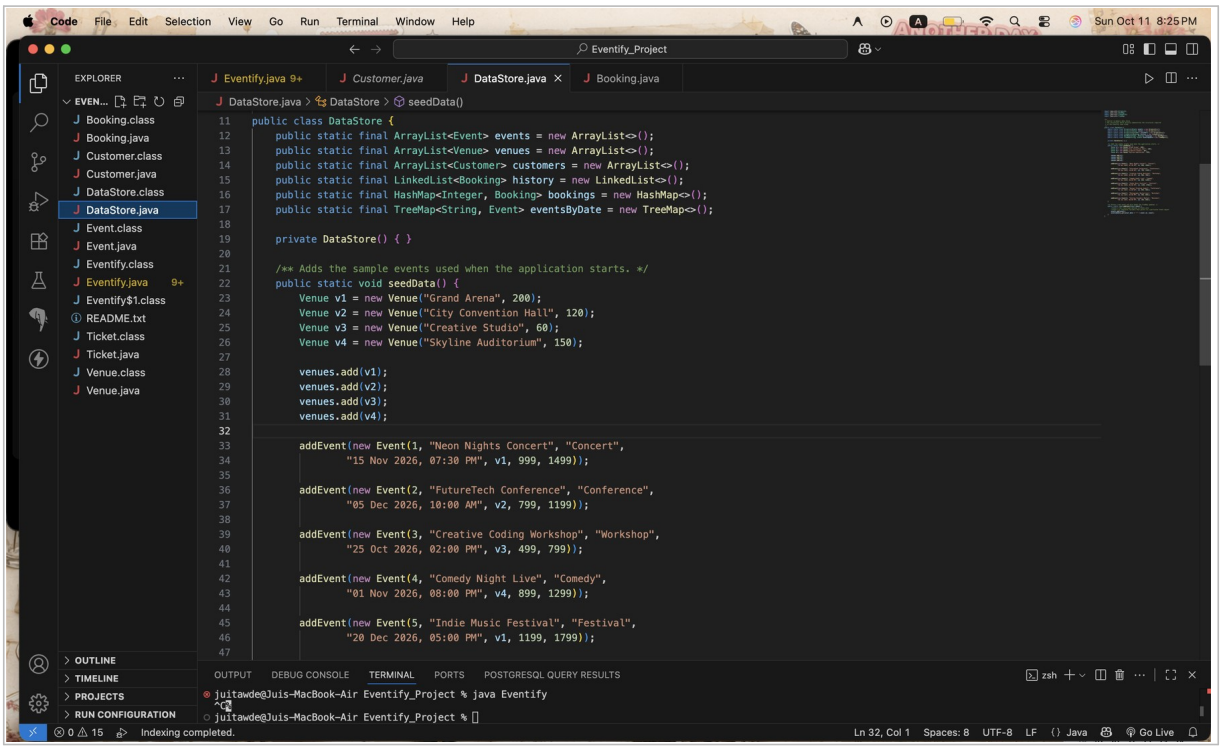


Figure 12: DataStore implementation showing the Java collections used to manage application data.

The DataStore class maintains the application's event, venue, customer, booking-history, active-booking, and date-keyed event collections. It also initializes the sample data used when the application starts.

Snapshot 13: Booking Logic

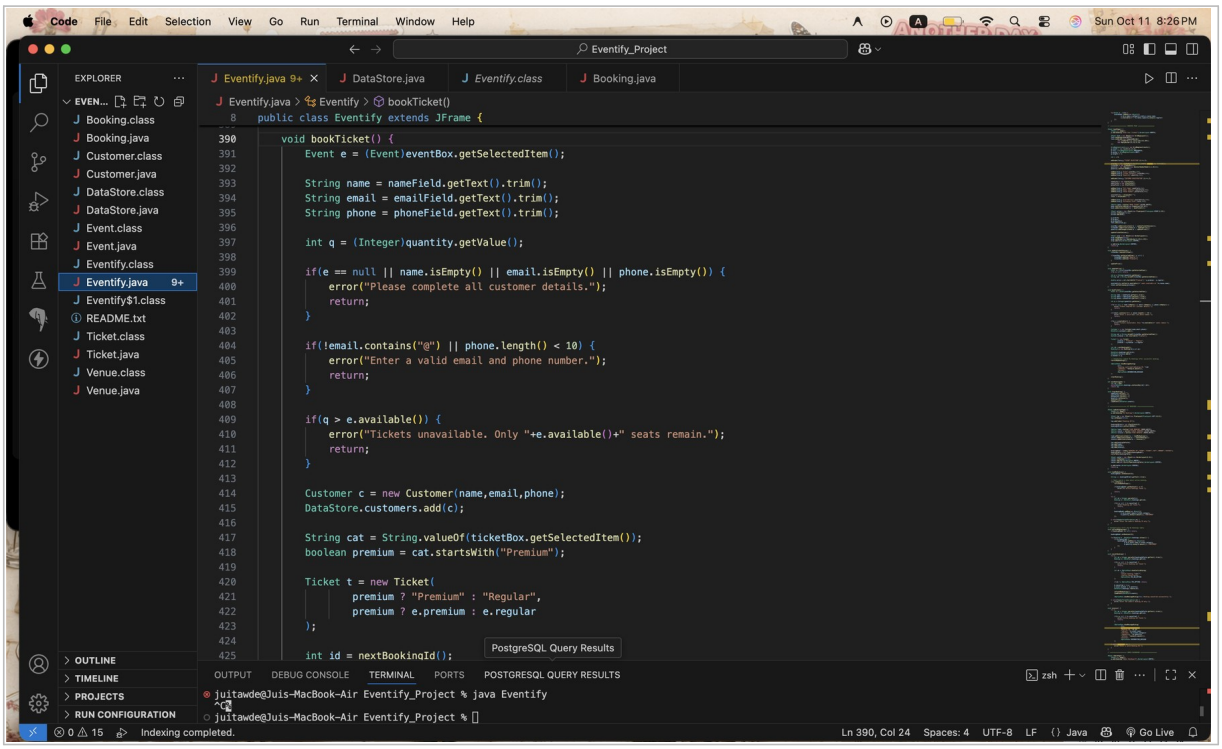


Figure 13: Booking implementation showing validation, object creation, collection updates, and seat availability management.

The booking method validates customer inputs, checks the available seats, creates the required objects, stores the booking, and updates the event's booked-seat count. It also refreshes the booking information and displays a confirmation message.

5.3 Summary of Implementation

The implementation demonstrates the integration of a graphical interface, object-oriented model classes, collection framework data structures, validation logic, searching, sorting, and booking management.

The screenshots show the application from both the user's perspective and the administrator's perspective. They also provide evidence of the implementation of the main booking workflow, invoice display, cancellation, event creation, and data management.

6. Problem Statement / Case Study Results and Conclusion

6.1 Results and Findings

The Eventify application successfully demonstrates the core operations of an event ticket booking and management system through a Java Swing interface.

The main findings are as follows:

1. **Event Display and Discovery:** The application displays event information in a structured table and supports searching by event name, category, or venue.
2. **Sorting:** Events can be sorted by date, name, and regular ticket price using Java's Comparator utilities.
3. **Ticket Booking:** Users can select events, choose ticket categories, specify the quantity, and enter customer details.
4. **Input Validation:** The booking process checks required fields, performs basic email and phone checks, and prevents bookings that exceed the currently available seats.
5. **Automatic Calculation:** The application calculates the estimated and confirmed booking amount using the selected ticket price and quantity.
6. **Booking Management:** Active bookings can be retrieved using booking IDs, and invoice details can be displayed through the interface.
7. **Cancellation:** The application supports cancellation of active bookings, updates booking status, and releases seats.
8. **Administrative Operations:** The administrator can view active bookings, access invoice details, cancel bookings, and add new events.
9. **Use of Data Structures:** ArrayList, LinkedList, HashMap, and TreeMap demonstrate how different collections can be selected according to their intended roles.
10. **Modular Design:** The use of separate model classes and a central DataStore class helps organize the application's information and responsibilities.

6.2 Limitations

Although the application demonstrates the main operations of event booking and management, certain limitations remain.

- The current version stores data in memory using Java collections. Event and booking information is not permanently saved to a database or file, so the data is not retained automatically after the application closes.
- The email and phone checks are basic validations and do not establish that the entered contact details are genuine.
- The application does not include an integrated online payment gateway.
- The administrative login is intended for the academic prototype and should not be treated as production-grade authentication.
- The application does not include multi-user synchronization, concurrent booking protection across different application instances, or a remote server.
- The current implementation does not provide advanced reporting or permanent historical records across application restarts.

These limitations identify possible areas for future improvement.

6.3 Future Scope

Eventify can be enhanced further through the following improvements:

1. **Database Integration:** Integrate a relational database such as MySQL or PostgreSQL to store events, venues, customers, tickets, and bookings permanently.
2. **Improved Authentication:** Introduce secure authentication, password hashing, role-based access, and appropriate access controls for administrative functions.
3. **Payment Integration:** Add a payment gateway to support online ticket payments and payment-status tracking.
4. **Enhanced Validation:** Improve email, phone number, date, price, and capacity validation.
5. **Reporting:** Add reports for event popularity, booking totals, cancellations, and revenue calculations.
6. **Persistent Booking History:** Maintain booking and cancellation records across application restarts.
7. **Multi-User Support:** Develop a client-server version with centralized data storage and safeguards against simultaneous bookings exceeding the available capacity.
8. **Improved User Experience:** Add event descriptions, venue details, filters, and more advanced booking-management features.

6.4 Conclusion

The Eventify case study demonstrates the development of a Java-based Event Ticket Booking and Management System using Java Swing and the Java collection framework.

The application integrates event discovery, searching, sorting, ticket selection, customer input validation, booking confirmation, invoice display, cancellation, administrative booking management, and event creation into a single graphical interface.

The project applies object-oriented programming by separating the main entities into model classes. It also demonstrates the practical use of ArrayList, LinkedList, HashMap, and TreeMap for organizing application data. Java event handling and layout managers are used to build an interactive desktop interface.

The implementation provides a functional foundation for understanding how a small event-management application can be designed and organized. Although permanent database storage, online payments, and production-level authentication are outside the current implementation, the project meets its core objective of demonstrating event ticket booking and management through a modular Java application.

Overall, Eventify provides practical experience in Java programming, Swing GUI development, collection framework usage, validation, sorting, and application design. It can serve as a foundation for developing a more advanced event ticketing system in the future.

7. References

1. Oracle. *Java Platform, Standard Edition Documentation.* Available at: https://docs.oracle.com/en/java/
2. Oracle. *Java Swing Documentation.* Available at: https://docs.oracle.com/javase/tutorial/uiswing/
3. Oracle. *The Java Collections Framework.* Available at: https://docs.oracle.com/javase/8/docs/technotes/guides/collections/
4. Oracle. *Class JFrame — Java Swing.* Available at: https://docs.oracle.com/javase/8/docs/api/javax/swing/JFrame.html
5. Oracle. *Interface Comparator — Java API Documentation.* Available at: https://docs.oracle.com/javase/8/docs/api/java/util/Comparator.html
6. Oracle. *Class HashMap — Java API Documentation.* Available at: https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html
7. Oracle. *Class TreeMap — Java API Documentation.* Available at: https://docs.oracle.com/javase/8/docs/api/java/util/TreeMap.html
8. Eventify project source code: Eventify.java, DataStore.java, and the model classes Event.java, Venue.java, Customer.java, Ticket.java, and Booking.java.